

Table of Contents

가시화 프로토타입 설명서

2026-08-28 참고 — 이 문서는 **기존 프로토타입 web-dashboard/**를 설명합니다. 이 앱은 그대로 동작하며 폐기하지 않습니다. 다만 최종 형태는 **탭 6개짜리 한 앱**이고, 아래 화면 일곱 개는 10월에 새 앱(viz-debugger/)의 **탭 ②~⑥으로 이식**됩니다 (구 임무 승인·진행과 구 판단·알림은 탭①로 통폐합·해체). 통합 후 구조는 요구사항정의서.md §3.1이 원천이고, 이식이 끝나면 이 문서를 **화면 여섯 기준으로 다시 씁니다**. 그때까지는 지금 동작하는 앱의 설명서로 유효합니다.

2026-08-25 개정 — 화면 7개를 기준으로 판단·알림과 노드 그래프를 추가했습니다. 노드 그래프의 F4 계약 무손실 왕복, 등록처 기반 카탈로그, 반영 그래프 역방향 관측과 목 실행기 경계를 반영했습니다. 개정 전 PDF는 _archive/에 있습니다.

0. 이게 뭔가

요구사항이 말로만 맞는지, 실제로 돌려도 맞는지를 **확인하기 위한 프로토타입**입니다. 화면을 예쁘게 만드는 것이 목적이 아니라, 요구사항 시트에 적은 주기·상태·경로가 **코드로 옮겼을 때도 성립하는지**를 보는 것이 목적입니다.

왜 지금 만들 수 있었나 — 목(mock) 게이트웨이

다른 파트의 구현이 아직 없습니다. 그런데 **기다리면 가시화는 아무것도 못 만듭니다**. 그래서 백엔드 게이트웨이가 있어야 할 자리에 **가짜 서버**를 하나 세웠습니다.

이 가짜 서버는 브라우저 안에 있지 않습니다. **별도 프로세스의 WebSocket 서버**로 돌아갑니다. 이게 중요한 이유는 —

- 브라우저 안에 가짜 데이터를 두면 **네트워크를 타지 않아서** 재연결·구독 복원·지연 같은 것을 검증할 수 없습니다
- 별도 프로세스로 두면 진짜 백엔드가 나왔을 때 **접속 주소만 바꾸면** 됩니다

그리고 이 가짜 서버는 **요구사항 시트의 주기 숫자를 그대로** 발행합니다. 로봇은 임무 중 50ms, 센서는 평시 1분, 카메라는 15fps, 액추에이터는 동작 중 100ms. 시트에 적은 숫자가 그냥 숫자가 아니라 **실제로 그 간격으로 데이터가 오는지**를 볼 수 있습니다.

무엇이 진짜고 무엇이 가짜인가

진짜

화면 코드 전부

데이터 레이어 (구독·병합·재연결·상태 보관)

WebSocket 왕복·HTTP 질의

주기·지연·상태 전이 규칙

파이프라인 계약 직렬화·검증

경계가 분명합니다. 가짜인 부분은 전부 목 서버 안에 있고, 화면 코드에는 가짜 데이터를 만드는 경로가 없습니다.

1. 켜는 방법

준비물

Node.js 22.6 이상이 필요합니다. 목 게이트웨이를 .ts 파일 그대로 실행하기 때문입니다.

node -v → v22.6.0 이상

없으면 <https://nodejs.org> 의 LTS를 설치합니다.

실행

```
cd web-dashboard
npm install
npm run dev
```

브라우저에서 **http://localhost:5173** 을 엽니다. (127.0.0.1이 아니라 localhost로 접속해야 합니다.)

npm run dev 하나로 **목 게이트웨이와 화면이 같이** 뜹니다. 따로 띄우려면 npm run dev:mock(서버만) / npm run dev:web(화면만).

안 켜질 때

가장 흔한 실패는 **포트 충돌**입니다.

[mock-gateway] 기동 실패 — 포트 8787 이 이미 사용 중이다.

이전 목 게이트웨이가 안 죽고 남아 있는 경우입니다. 터미널을 Ctrl+C 없이 닫으면 이렇게 됩니다. README.md의 “안 켜질 때 — 포트 충돌”에 종료 방법이 있습니다.

상황을 만들어 보는 방법 — 시나리오

그냥 켜두면 평온한 상태만 보입니다. 장애·지연·실패를 보려면 **시나리오를 재생**합니다.

```
npm run scenario <이름>
```

예: npm run scenario camera-silence → 60초 뒤 카메라가 “판단 불가”로 바뀝니다.

전체 목록은 5장에 있습니다.

2. 화면 일곱 — 무엇을 보면 되나

상단 탭으로 전환합니다. 탭을 옮겨도 구독이 끊기지 않습니다 — 돌아왔을 때 화면이 비어 있으면 안 되니까요.

2-1. 구역 현황판

관련 요구사항 VZ-U-01 · VZ-I-01 · VZ-I-02 · VZ-I-03

무엇을 보는 화면인가 — 구역 안의 장치가 지금 어떤 상태인지를 카드로 늘어놓습니다.

여기서 확인할 것은 상태가 넷이라는 점입니다.

표시

정상

장애

의도적 미배포

판단 불가

“장애”와 “판단 불가”를 가르는 것이 이 화면의 핵심입니다. 둘을 하나로 합치면 “고장난 것”과 “연락이 안 되는 것”이 같아 보이는데, 관제에서 이 둘은 완전히 다른 조치를 부릅니다.

그래서 상태를 하나로 뭉쳐서 저장하지 않고 **세 층(기기 자기보고 / 서버 판정 / 배포 여부)**을 각각 보관하고 표시값만 파생시킵니다.

꼭 봐야 할 것

- robot-03 — **값을 한 번도 발행하지 않는데 카드가 보입니다.** 레지스트리(존재해야 할 목록)를 근거로 그리기 때문입니다. 값이 오는 것만 그리면 “있어야 할 게 안 왔다”를 영영 못 봅니다
 - 새로고침 — **빈 카드가 없습니다.** 구독하는 순간 백엔드가 마지막 값을 한 번 던져주기 때문입니다. 이게 없으면 센서는 최대 1분간 빈 화면입니다
 - npm run scenario camera-silence → 60초 뒤 **판단 불가**로 바뀝니다. 이때 기기 자기 보고는 마지막 값(ok)이 그대로 남아 있습니다 — 두 층이 서로 다른 주체의 말이라는 증거입니다
-

2-2. 제어 패널

관련 요구사항 VZ-O-01 · VZ-O-02 · VZ-O-05 · VZ-I-05 · VZ-C-04

무엇을 보는 화면인가 — 수문을 열고 닫고, 그 결과가 어떻게 돌아오는지를 봅니다.

여기서 확인할 것은 “눌렀다”와 “됐다”가 다르다는 점입니다.

명령은 네 단계로 돌아옵니다 — **수신 확인** → **수행 중** → **물리 상태 변화** → **완료**. 화면은 이걸 세 가지(진행중·확정·실패)로 보여주되, **수신 확인을 확정으로 치지 않습니다**. 수문은 되돌리기 어려우니 “접수했다”가 아니라 “실제로 움직였다”를 확인해야 합니다.

두 개의 키

명령을 누르면 화면이 **요청 식별자**를 붙여 보내고, 백엔드가 **상관키**를 발급해 내려줍니다. 왜 둘이냐면 —

- 상관키는 백엔드가 만드는데, **그게 도착하기 전 구간**에도 버튼을 잠그고 진행 표시를 걸어야 합니다
- 그 구간에 걸 대상이 요청 식별자입니다

화면에는 이 두 키가 **하나의 상태로 보입니다**. 키가 몇 개인지는 데이터 레이어 안에서 끝납니다.

꼭 봐야 할 것

- `npm run scenario ack-late` — 상관키가 **진행 이벤트보다 늦게** 도착합니다. 그 사이에 온 단계들이 **보류됐다**가 **흡수**되고, “보류 후 흡수” 표시가 붙습니다. 순서를 믿는 코드는 실제 백엔드에서 깨지기 때문에 일부러 만든 경로입니다
 - `npm run scenario ack-drop` — 상관키가 **끝내 안 옵니다**. 화면은 “만료 — 실행 여부를 이 화면이 알 수 없다”고 **정직하게** 표시합니다
 - `npm run scenario control-lock` — 통신이 끊기면 **버튼이 잠기고 사유가 보입니다**. `control-unlock` 후에도 서버가 실제 상태를 재확인할 때까지 잠금이 유지됩니다
 - `npm run scenario role-narrow` — 역할을 503 구역 담당으로 좁히면 **다른 구역 수문의 버튼이 잠깁니다**. 화면에 “잠금 우회해 발행” 버튼이 있는데, 눌러 보면 **서버가 거부합니다**. 화면 차단은 편의일 뿐 실제 방어선은 백엔드라는 것을 눈으로 보는 장치입니다
 - **마지막 조작자 패널** — 열 때만 조회하고 주기 폴링을 하지 않습니다. 개발자 도구 네트워크 탭에서 확인할 수 있습니다
-

2-3. 임무 승인·진행

관련 요구사항 VZ-U-07 · VZ-U-05

무엇을 보는 화면인가 — AI가 만든 계획을 사람이 승인하고, 그 계획이 어디까지 갔는지를 봅니다.

여기서 확인할 것은 승인 전에는 아무것도 실행되지 않는다는 점입니다.

AI가 계획을 만들고 검증까지 마쳐도, **사람이 승인 버튼을 누르기 전까지 로봇은 움직이지 않습니다**. 목 서버가 승인 없이는 진행 이벤트를 아예 보내지 않도록 만들어 놔습니다.

근거를 펼쳐 볼 수 있습니다 — 어떤 임무에서 나왔는지, 어느 구역을 어떤 순서로 지나는지, 어떤 검증을 통과했는지, 어느 버전의 생성기가 만들었는지.

누가 가져다주는가 — 계획을 만드는 것은 AI지만, **가시화에 가져다주고 승인 결과를 받아 가는 것은 백엔드**입니다. 화면에 중계 경로가 표시되어 AI 산출 구간과 백엔드 중계 구간이 구분

됩니다. 나중에 승인이 안 먹었을 때 “AI가 계획을 못 만든 것”인지 “중계가 끊긴 것”인지를 갈라야 하기 때문입니다.

꼭 봐야 할 것

- `npm run scenario plan-propose` — 계획이 승인 대기로 내려옵니다. 승인하면 **한 박자 뒤에** 실행이 시작됩니다(백엔드 중계 구간). 거부하면 사유가 같은 경로로 돌아갑니다
 - `npm run scenario plan-propose-failing` — **구간 4/5에서 실패**합니다. 어느 단계에서 왜 실패했는지 펼쳐 볼 수 있습니다
-

2-4. 지표 조회

관련 요구사항 VZ-I-04 · VZ-C-03

무엇을 보는 화면인가 — 수위·유속 같은 시계열을 그래프로 봅니다.

여기서 확인할 것은 평시에 받는 값이 원본이 아니라는 점입니다.

설계상 **raw**는 엣지에 남고 백엔드에는 **구역 요약만** 올라옵니다. 장치가 늘어도 백엔드 부하가 장치 수가 아니라 구역 수에 비례하게 만드는 구조입니다.

그래서 화면에 오는 값에는 **“요약 · 구역 · 15초”** 같은 표기가 붙습니다. 이 표기가 없으면 화면이 요약값을 원본으로 알고 **또 평균을 냅니다**. 그러면 숫자가 틀리는데 **그래프는 멀쩡해 보입니다**. 발견이 늦는 종류의 오류라 아예 계산을 막아 났습니다.

원본이 필요하면 따로 요청합니다. “원본 보기”로 전환하면 질의 프록시가 엣지로 중계해 가져옵니다 — **느립니다**. 실측으로 요약은 12ms에 60점, 원본은 975ms에 900점이었습니다. 이 느림은 연출이 아니라 실제 엣지 중계를 흉내 낸 것이고, 화면이 “가져오는 중”을 그릴 이유가 여기서 나옵니다.

꼭 봐야 할 것

- 그래프 위의 **요약 표기** — 지금 보는 게 요약인지 원본인지
 - 요약값에 평균을 적용해 보면 **계산이 수행되지 않고** 차단 사유가 뜹니다
 - `npm run scenario agg-unlabeled` — 집약 표기가 **깨진 채로** 옵니다. 화면은 이것을 원본으로 단정하지 않고 **“표기 불명”** 으로 두고 계산을 막습니다. 모르는 것을 원본으로 취급하면 보호 장치가 조용히 꺼지기 때문입니다
 - `npm run scenario agg-normal` — 정상 표기로 되돌립니다
-

2-5. 영상 오버레이

관련 요구사항 VZ-I-06 · VZ-I-07 · VZ-I-09

무엇을 보는 화면인가 — 영상 위에 AI 탐지 결과(박스)를 겹쳐 그립니다.

이 화면이 프로토타입에서 값이 제일 큼니다. 그림으로만 주장하던 것을 숫자로 바꿔 놓았기 때문입니다.

문제는 이렇습니다. AI가 프레임을 보고 결과를 내는 데 시간이 걸립니다. 0.2초 걸린다면, 결과가 도착할 때 화면은 이미 **3프레임쯤 앞서 있습니다**(15fps 기준). 그 결과를 그냥 지금 화면에 그리면 **박스가 대상을 못 따라갑니다.**

해결은 결과에 **“이건 몇 번 프레임을 본 결과다”** 를 실어 보내는 것입니다. 화면은 그 프레임에 맞춰 그립니다.

정합 on/off 토글이 있습니다.

설정

정합 **컴**

정합 끄 · 지연 200ms

정합 끄 · 지연 500ms

2026-08-24 재측정 (커밋 09add2c 기준). 앞선 판의 46.8px·102.3px는 **인지 출처를 나누기 전** 수치라 지금 화면과 맞지 않습니다.

지연을 2.5배로 늘리면 엣지 쪽 어긋남이 2.3배가 됩니다. 프레임 참조가 왜 계약에 필요한지를 이 표 하나로 설명할 수 있습니다.

온디바이스 값이 지연과 무관하게 30px 안팎인 이유는 온보드 판단이 60ms로 고정돼 있기 때문입니다. vision-delay-*는 **엣지 추론 지연만** 바꿉니다 — 안전 판단이 엣지 왕복을 기다리지 않는다는 것이 여기서도 드러납니다.

꼭 봐야 할 것

- 토글을 껐다 켜보면 박스가 대상에서 뒤쳐지는 것이 **눈에 보이고**, 뒤쳐진 픽셀 거리가 화면에 숫자로 뜹니다
- `npm run scenario vision-delay-500` — 어긋남이 커집니다
- `npm run scenario vision-bbox-normalized / vision-bbox-absolute` — 좌표 형식을 바꿔도 **박스가 제자리에** 옵니다
- 신뢰도가 낮은 탐지는 **시각적으로 구분**됩니다. 확실한 것과 애매한 것이 똑같이 보이면 안 되니까요
- 온디바이스의 진행영역·접근 변화와 엣지의 정밀 분류·추적은 **출처별로 다른 색과 표기**를 씁니다
- vision-edge-off에서는 기본 온디바이스 판단은 남고 정밀 인지만 사라집니다. vision-link-off에서는 관측 소스별 추적을 유지합니다
- 참조 프레임이 버퍼에 없으면 **참조 프레임 없음**을 표시하며, 현재 프레임과 비교한 수치를 정합 결과로 주장하지 않습니다
- 탭을 떠나면 **프레임 루프가 멈춥니다**. 안 보는 영상을 계속 그리면 렌더 예산을 낭비합니다

영상은 합성입니다. 실제 픽셀은 업무·관측과 분리된 미디어 평면으로 가야 합니다 (AI-C-14). GStreamer 어댑터의 AI 입력 경로는 생겼지만 뷰어용 출력 분기와 담당이 아직 정해지지 않아, 목 서버가 프레임 번호가 찍힌 도형 영상을 15fps로 내려 줍니다. 정합 검증 목적에는 충분합니다.

2-6. 판단·알림

관련 요구사항 VZ-I-08 · VZ-I-10 · VZ-O-04 · VZ-U-03

무엇을 보는 화면인가 — AI 판단의 결과만 보여주는 것이 아니라, 어떤 입력과 규칙으로 그 판단에 도달했는지와 가시화 자체가 제때 처리되고 있는지를 함께 봅니다.

같은 데이터를 보더라도 역할에 따라 필요한 깊이가 다릅니다. 운영자는 지금 조치할 내용을, 분석자는 근거와 규칙을, 결심자는 권고와 영향 범위를 먼저 봅니다. 탭이나 역할을 바꾼다고 구독을 새로 만들지 않고 **표시 깊이만 바꿉니다.**

꼭 봐야 할 것

- 위험 단계와 권고 옆의 **근거 펼치기** — 입력값·규칙·판정 사유를 되짚을 수 있습니다
 - AI 결과가 없거나 실패했을 때 정상처럼 빈칸으로 두지 않고 **판단 불가·실패 상태**를 드러냅니다
 - 클라이언트 처리 지연은 장치 상태와 섞지 않고 별도 자체 관측 지표로 보냅니다
 - 경고는 닫을 수 있지만 원인이 해소되지 않은 사실까지 지워지지는 않습니다
-

2-7. 노드 그래프

관련 요구사항 VZ-U-04 = REQ-1001~REQ-1007

무엇을 보는 화면인가 — 기본 모드에서는 임무→Sub task→실행·영상·센서 근거를 한 장에 연결하고, 보조 모드에서는 source→transform→sink 데이터 해석 파이프라인을 편집합니다.

임무 관제 모드

계획이 도착한 대상과 같은 구역의 근거 후보를 **레지스트리에서** 구성합니다. 특정 robot-01이나 네 장치 목록을 화면 코드에 고정하지 않으므로 장치와 임무 대상이 늘면 선택지와 근거 노드가 함께 늘어납니다. Sub task의 대기·진행·완료·실패·건너뛴 상태와 근거 연결을 한 화면에서 볼 수 있습니다.

데이터 파이프라인 모드

- 노드를 놓고 포트를 연결하면 방향·타입·필수 입출력·순환·고립 여부를 검사합니다
- 초안은 { pipeline, layout }입니다. pipeline은 contracts/pipeline.schema.json 그대로이고 좌표는 계약을 오염시키지 않도록 바깥 layout에 둡니다
- **계약 JSON 보기**에서 좌표 없는 F4 계약을 확인할 수 있습니다. 같은 계약을 다시 읽으면 별도 레이아웃과 합쳐 같은 화면으로 복원됩니다
- 노드 목록은 코드 상수가 아니라 contracts/examples/의 등록 디스크립터·렌더러·데이터소스에서 만들어집니다. 예시 파일을 하나 추가하면 TypeScript를 고치지 않아도 카탈로그가 늘어납니다
- 시험 실행 뒤 받은 토큰이 있어야 운영에 반영할 수 있고, 반영 뒤에는 직전 버전으로 되돌릴 수 있습니다

시험 결과와 운영 관측은 다릅니다. 시험 결과의 rows.elapsed_ms는 초안에 대한 **목 실행 결과**입니다. 별도 **운영 그래프 관측** 표는 반영된 그래프의 노드별 최근 수신 시각·건수·마지막 값

과 붙은 대상을 보여줍니다. source는 게이트웨이가 실제로 받은 목 장치 봉투를 관측하지만 transform·sink는 목 실행기가 전달한 값입니다. 화면도 둘을 다른 영역과 문구로 구분하며, 실물 실행기나 영속 관측인 것처럼 보이지 않습니다.

꼭 봐야 할 것

- 임무 관제에서 정상 임무와 4번 Sub task 실패 임무를 재생해 진행·실패 전파 확인
 - 데이터 파이프라인에서 잘못된 연결을 시도해 구체적인 거부 사유 확인
 - **계약 JSON 보기**에서 노드 좌표가 계약 본문에 없는지 확인
 - 시험 실행 → 운영 반영 뒤 **모의 시험 결과**와 **운영 그래프 관측**이 분리되어 있는지 확인
 - 새 시험 없이 바뀐 초안을 반영할 수 없는지, 직전 버전 되돌리기가 되는지 확인
-

3. 눈에 안 보이지만 중요한 것

화면에 나타나지 않지만 요구사항의 핵심인 것들입니다.

3-1. 데이터는 전량 받고, 화면 반영만 묶는다

로봇이 초당 20번 값을 보냅니다. 받을 때마다 화면을 다시 그리면 **오히려 버벅입니다**. 그렇다고 데이터를 버리면 이동 궤적이 끊깁니다.

그래서 **데이터는 전부 받되 화면 반영만 100ms 창으로 묶습니다**. 초당 10회를 넘지 않습니다.

3-2. “값이 오래됐다”는 판정은 서버가 한다

화면이 마지막 수신 시각으로 계산하면 **사용자 PC 시계에 의존**하게 됩니다. 시계가 틀린 PC에서는 멀쩡한 장치가 판단 불가로 뜹니다. 그래서 판정은 서버가 하고 화면은 결과만 받습니다.

3-3. 감사는 화면이 쓰지 않는다

“누가 언제 무엇을 시켰나”는 화면이 직접 기록하지 않고 **필드만 실어 보냅니다**. 이유가 셋입니다 —

1. **위조** — 화면이 직접 쓰면 “누가 했다”를 클라이언트가 자기 입으로 말하는 셈입니다
2. **시계** — 장치는 NTP로 시각을 맞추지만 사용자 PC에는 그런 보장이 없습니다
3. **유실** — 별도 경로로 보내면 탭을 닫는 순간 “실행은 됐는데 기록이 없음”이 생깁니다

3-4. 끊기면 스스로 붙고, 구독을 되살린다

목 서버를 껐다 켜면 화면이 **자동으로 재접속**하고 **원래 구독을 복원**합니다. 복원되면 현재값이 다시 채워져서 화면에 공백이 없습니다.

4. 지금 확인할 수 있는 것 / 아직 없는 것

확인할 수 있는 것

- 상태 4종이 실제로 구분돼 표시되는가 → **된다**
- 요구사항에 적은 주기대로 데이터가 오는가 → **온다** (50ms · 1분 · 15fps · 100ms · 15초)
- 프레임 참조가 없으면 박스가 얼마나 어긋나는가 → **숫자로 나온다** (옛지 200ms에 91.8px · 500ms에 210.0px)
- 승인 없이 계획이 실행되는가 → **안 된다**
- 권한 범위 밖 명령이 화면을 우회해도 막히는가 → **막힌다**
- 요약값을 다시 평균 내려 하면 어떻게 되는가 → **계산이 수행되지 않는다**
- 서버를 껐다 켜면 화면이 복구되는가 → **된다**
- 재접속 때 캐시 금지 채널이 재생되지 않고 원본 시각이 보존되는가 → **자동 검증을 통과했다**
- 같은 게이트웨이 계약으로 3D 공간에 위치를 그리는가 → **Unity 뷰어에서 된다**
- 노드 편집기 초안이 좌표 없이 F4 계약으로 왕복하는가 → **자동 검증을 통과했다**
- 등록 파일만 추가해도 노드 카탈로그가 늘어나는가 → **자동 검증을 통과했다**
- 반영된 파이프라인에서 지금 흐르는 값을 노드별로 볼 수 있는가 → **된다** (하류는 목 전달 값)

아직 없는 것

없는 것

음성 명령

LLM 그래프 초안 생성

실제 카메라 영상

실제 백엔드·파이프라인 실행기 연결

Unity 시뮬레이터

5. 시나리오 전체 목록

npm run scenario <이름>

이름	무엇이 일어나나
camera-silence	camera-02 침묵 → 60초 뒤 판단 불가
camera-resume	camera-02 발행 재개 → online 복귀
sensor-offline	sensor-02 연결 끊김 → offline 후 복구
sensor-surge	sensor-01 급변 → 즉시 발행 + 이벤트 모드(1초)
command-fail	다음 명령 1건을 실패시킴
ack-late	상관키를 진행 이벤트보다 늦게 보냄 → 보류

ack-drop	후 흡수
agg-unlabeled	상관키를 아예 안 보냄 → 만료 정리
	집약 표기에서 종류를 빼고 발행 → 표기 불명 처리
agg-odd-string	집약 표기를 계약 밖 형태로 발행
agg-normal	집약 표기를 정식 계약값으로 되돌림
role-narrow	역할을 503 구역 담당으로 좁힘
role-full	역할을 전 구역으로 되돌림
control-lock	actuator-01 통신 두절 → 제어 잠금
control-unlock	actuator-01 복구 → 재확인 후 해제
plan-propose	계획 하나를 승인 대기로 내려보냄
plan-propose-failing	구간 4/5에서 실패하는 계획
vision-delay-200	추론 지연 200ms (기본)
vision-delay-500	추론 지연 500ms → 어긋남 확대
vision-bbox-normalized	bbox를 정규화 좌표(0~1)로
vision-bbox-absolute	bbox를 픽셀 절대 좌표로
vision-inference-320 / vision-inference-640	추론 해상도 전환 — 표시 해상도와 달라도 박스가 제자리인지
robot-idle / robot-mission	robot-01 임무 토글 — 50ms(20Hz) ↔ 5초
vision-edge-off / vision-edge-on	선택형 엣지 정밀 인지를 제거/복구
vision-link-off / vision-link-on	다중 관측 연계를 제거/복구
cache-policy-audit	캐시 정책 선언과 실제 캐시를 대조
command-roundtrip-slow / command-roundtrip-zero	명령 왕복 지연 주입/해제

6. 처음 보는 사람을 위한 5분 코스

1. npm run dev → **http://localhost:5173**
2. **구역 현황판** — robot-03이 값 없이도 카드로 있는 것 확인. 새로고침해도 빈 카드가 없는 것 확인
3. npm run scenario camera-silence → 60초 기다렸다가 **판단 불가** 확인
4. **제어 패널** — 수문 열기. 진행중 → 확정으로 바뀌는 것과, 수행 중 진행 상태가 보이는 것 확인
5. npm run scenario role-narrow → 다른 구역 수문 버튼이 잠기는 것 확인. “**잠금 우회해 발행**” 눌러서 서버가 거부하는 것 확인
6. **영상 오버레이** — 정합 토글을 껐다 켜면서 **뒤쳐진 픽셀 수**가 화면에 뜨는 것 확인
7. npm run scenario vision-delay-500 → 그 숫자가 커지는 것 확인

- 8. **판단·알림** — 역할을 바꿔도 구독은 유지되고 설명 깊이만 달라지는지 확인
- 9. **노드 그래프** — 임무 실패 전파를 본 뒤 데이터 파이프라인에서 시험 실행→반영→운영 관측 확인

6·7번은 영상 정합의 필요성을 숫자로 보여주고, 9번은 계약·편집·운영 관측의 경계를 한 번에 보여줍니다.